Executable Scientific Contracts for Verifiable LLM and Agent Research

Amir Reza Peimani
University of Toronto
amir.peimani@utoronto.ca

Abstract

Research audits often uncover code discrepancies whose effects on reported results remain unclear. We study executable scientific contracts: checks that connect a paper claim, its implementation, a controlled input, and evidence from released artifacts. We examine 25 checks across five agent and language model repositories. In tests of upstream implementations, the contracts detected all 17 faulty variants, accepted all three repairs, and raised no flags on 12 harmless rewrites. In AgentPRM, correcting a temporal error changes state targets and preference construction while preserving the mean over complete trajectory visits. ToolACE's crop defect is inactive across 9,964 examples produced by the official conversion. In AgentAbstain, the released task composition permits a maximum difference of 2.4125 percentage points between two weighting conventions. These cases show how executable checks distinguish implementation errors, inactive defects, and unspecified evaluation choices, giving reviewers reproducible evidence for assessing an audit finding.

1 Introduction

Language models increasingly assist scientific writing, code inspection, and peer review. An audit finding still needs a precise connection to the paper's computation and data. We study checks that make this connection executable and show which conclusions the available artifacts support.

Reproducibility taxonomies distinguish repeatability and reconstruction [Tatman et al., 2018]. Algorithmic unit tests and mutation testing examine implementation behavior and the ability of tests to detect faults [Raff and Holt, 2023, Just, 2014, Panichella and Liem, 2021, Tufano et al., 2019]. Guidelines for agent benchmarks also address evaluation quality [Zhu et al., 2025].

RepoAudit uses LLM agents to audit repositories [Guo et al., 2025]. SciCoQA evaluates scientific paper–code alignment using verified mismatch reports [Baumgärtner and Gurevych, 2026]. AuditOwl extracts paper claims, traces them to code, and verifies candidate discrepancies with executable checks [Anonymous authors, 2026]. SANA generates contracts for scientific data-analysis workflows [Vu et al., 2026].

Rules to Tools supplies executable scientific checks for coding agents [Ning et al., 2026]. ReproAgent maintains implementation requirements during paper-to-code reproduction [Hu et al., 2026]. Semantic Mutation Score evaluates test adequacy using mutations based on scientific semantics [Li et al., 2026].

Following design by contract [Meyer, 1992], we record an audit finding as an executable check of the stated behavior. Each contract connects the claim, implementation, test input, activation in released data, and evidence about the reported result. We examine 25 checks across five repositories and evaluate how the checks respond to faults, repairs, and harmless changes.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: AI-Native Academia: Authorship, Peer Review, and Conference Governance under AI.

2 Executable scientific contracts

Contract contents. A contract records the statement or equation being checked, the implementing code, an input that distinguishes expected from implemented behavior, whether the relevant condition occurs in released data, and the evidence connecting the finding to a reported result. We organize these records as

$$\text{claim} \rightarrow \text{implementation} \rightarrow \text{controlled check} \rightarrow \text{activation} \rightarrow \text{result evidence}.$$

Outcome labels. Conformant means that the implementation matches the stated expectation on the tested input. Discrepant means that the check exposes a mismatch. Inactive means that a demonstrated defect is not triggered in the inspected released data. Convention sensitivity means that the source leaves a choice unspecified and reasonable alternatives produce different values. We record implementation behavior, eligible examples, activation, and the connection to a reported result separately.

Case selection. We retained five checks for each of AgentPRM [Choudhury, 2025], ContractBench [Wang et al., 2026], ToolACE [Liu et al., 2024], τ-bench [Yao et al., 2025], and AgentAbstain [Liu et al., 2026]. The first four cases revisit earlier audit findings. Their 20 checks were constructed with those findings already known. The records do not specify a stopping rule for the retrospective selection. The five AgentAbstain checks were specified before execution. The inventory includes substantive checks, consistency checks, and summaries, covering equations, schemas, logging, aggregation, and data support. Repository revisions and the complete inventory appear in the supplement.

Selecting a new contract. We recommend starting with a claim central to a reported result. Locate the implementing computation and construct an input that distinguishes the stated behavior from a plausible error. Then check whether that input condition occurs in the released data.

Implementation tests. The checks execute pinned upstream functions or unchanged function extractions whose hashes are recorded. We compare field/type validation, the original adapters, and contracts that test function behavior. The original adapters combine source inspection with locally reconstructed calculations. The faults preserve the required fields and types. The original-adapter comparison covers AgentPRM and ToolACE; relevant native test coverage was unavailable for the selected behaviors.

The experiment contains 60 cases: 17 faulty variants, comprising three original faults and 14 injected variants; three repairs; 12 harmless rewrites; eight conformant original cases; and 20 cases with missing evidence or ineligible inputs. A shared check returns 'not evaluated' when required evidence is missing or no eligible examples are available, before the compared methods run. We reran the same 60 cases after correcting the eligibility checks and prediction-mask accounting.

After freezing the corrected checks, we constructed nine additional cases covering terminal-reward remapping, response-token order, and replacement of the outer scenario mean with a median. Each family contains a fault, a reference implementation, and a harmless rewrite. The checks and their imports remained unchanged. These nine cases are reported separately.

The earlier mutation experiment exercised four fixture validators. Its 30 mutations and the eight externally motivated operators are documented in the supplement.

Complete executable example. The AgentAbstain conjunction check uses the four Boolean act/abstain pairs, for which paired accuracy is 0.25. The listing constructs the evaluator input and calls `compute_summary_metrics` from revision f5812497. Code, study artifacts, environment specifications, and run commands are available at `https://github.com/AmirRezaPeimani/executable-scientific-contracts`.

```
#!/usr/bin/env python3
import argparse
import hashlib
import inspect
import subprocess
import sys
from pathlib import Path
import pandas as pd

parser = argparse.ArgumentParser()
parser.add_argument("--source-root", type=Path, required=True)
root = parser.parse_args().source_root.resolve()
pin = "f581249704b26804e28a39e37396f1be00b71a4d"
revision = subprocess.check_output(
    ["git", "-C", str(root), "rev-parse", "HEAD"], text=True).strip()
assert revision == pin
sys.dont_write_bytecode = True
sys.path.insert(0, str(root))
from eval.statistics.analysis import compute_summary_metrics
source = Path(inspect.getsourcefile(compute_summary_metrics)).resolve()
assert source == root / "eval/statistics/analysis.py"
cells = ((True, True), (True, False), (False, True), (False, False))
records = []
for pair, outcomes in enumerate(cells):
    for task_type, passed in zip(("act", "abstain"), outcomes):
        records.append(
            {"provider": "fixture", "model": "fixture", "category": "boolean",
             "action_type": "operational", "metric_name": "combined",
             "task_id": str(pair), "task_type": task_type, "pass": passed})
summary = compute_summary_metrics(pd.DataFrame(records))
paired = float(summary.iloc[0]["paired_accuracy"])
assert paired == 0.25
assert int(summary.iloc[0]["num_pairs"]) == 4
print("paired_accuracy: 0.25 PASS")
print("source_sha256:", hashlib.sha256(source.read_bytes()).hexdigest())
```

The check prints ‘paired_accuracy: 0.25 PASS’ and the SHA-256 hash of the imported source file.

3 Results

Implementation tests. Figure 1 compares decisions on faulty implementations, repairs, and harmless rewrites. Acceptance of repaired code and harmless rewrites was the clearest improvement over the original adapters. The field/type checks accepted the faulty outputs because their fields and types were preserved. The 20 cases with missing evidence or ineligible inputs exercised the shared check that returns ‘not evaluated’.

The frozen checks also distinguished the faults, reference implementations, and harmless rewrites in the nine additional cases. Table 1 reports these results separately.

Faults, repairs, and harmless changes
Each fraction counts evaluated cases.

	Field / type validation	Original adapters	Behavioral contracts
AgentPRM			
Faulty variants detected	0/4	3/4	4/4
Genuine repairs accepted	2/2	1/2	2/2
Harmless rewrites flagged	0/2	1/2	0/2
Missing / ineligible: not evaluated	2/2	2/2	2/2
ToolACE			
Faulty variants detected	0/5	5/5	5/5
Genuine repairs accepted	1/1	0/1	1/1
Harmless rewrites flagged	0/2	2/2	0/2
Missing / ineligible: not evaluated	2/2	2/2	2/2
AgentAbstain			
Faulty variants detected	0/8	Unavailable	8/8
Genuine repairs accepted	n/a	n/a	n/a
Harmless rewrites flagged	0/8	Unavailable	0/8
Missing / ineligible: not evaluated	16/16	Unavailable	16/16

Blue: expected decision. Orange: miss or false alarm. Gray: coverage unavailable.

Figure 1: Decisions on 52 of the 60 implementation cases. The eight conformant original cases are reported in the supplementary results table. Blue marks the expected decision, orange a missed fault or false alarm, and gray unavailable coverage.

Table 1: Nine cases constructed after the checks were frozen. Each mutation family contains a fault, a reference implementation, and a harmless rewrite.

Method	Faults detected	References accepted	Benign flags	Uncovered faults
Field/type	0/3	3/3	0/3	0
Native tests	Unavailable	Unavailable	Unavailable	3
Original adapters	2/2 covered	0/2 covered	2/2 covered	1
Behavioral contracts	3/3	3/3	0/3	0

All nine cases completed and were included. The checks and their imports matched the recorded hashes.

AgentPRM: temporal targets. AgentPRM's target equation discounts a terminal reward by remaining trajectory length, while both target callers pass the forward time index. For a successful four-action trajectory at the released $\gamma = 0.95$, correcting the callers changes the signed targets from $(1, 0.95, 0.9025, 0.857375)$ to $(0.857375, 0.9025, 0.95, 1)$. The mean remains 0.92746875, but the target assigned to each state changes.

Reversal preserves the multiset over complete trajectory visits. Averaging deduplicated state-action targets can give different means before and after repair. The absolute-target exporter applies $0.5(Q + 1)$; preference construction uses the signed values. For two successful continuations from a shared state, with lengths four and eight, the signed margin changes from zero to $0.95^3 - 0.95^7$, exceeding the released preference threshold $\delta = 0.03$. A preference is added for each of the 28 chosen length pairs. Figure 2 shows how target assignment changes with trajectory position and length.

The rollout also copies observations into the action field of the query history. A test agent records six incorrect history entries before repair and none afterward. The separately saved trajectory contains the executed actions in both versions. Historical training traces were unavailable.

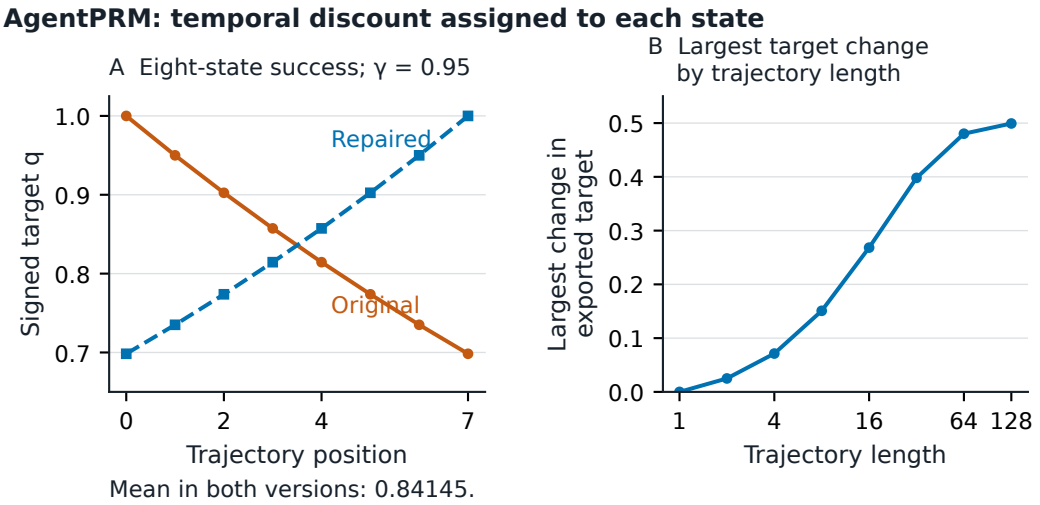


Figure 2: AgentPRM targets before and after correcting the discount exponent. The left panel shows signed targets for an eight-state successful trajectory at $\gamma = 0.95$. The right panel shows the largest change in exported targets across trajectory lengths. Exported targets use $0.5(Q + 1)$, giving half the change in signed targets.

ToolACE: cropping and released data. ToolACE's loader constructs the response mask using coordinates from before left cropping. In the controlled example, cropping a sequence of ten prompt tokens and four response/EOS tokens to eight inputs removes all supervision under the original mask. Correcting the coordinates restores four supervised predictions. Figure 3 shows the alignment between retained inputs, next-token labels, and mask positions.

The official conversion produces 9,964 examples. Running the loader with the recorded tokenizer finds no sequence above the 8,192-token limit; the maximum length is 3,945. The crop defect is therefore inactive on these converted examples. The supplement reports the conversion counts, excluded candidates, and differences from the earlier custom converter.



Figure 3: ToolACE mask alignment after left cropping. Ten prompt tokens and four response/EOS tokens are cropped to eight inputs, removing six prompt tokens. The mask at position i weights the prediction of retained token $i + 1$. The original mask suppresses all four response/EOS predictions; the corrected mask retains them. Padding and the final position with no next-token prediction are excluded.

ContractBench: released evaluation records. ContractBench reports 56.6% for Qwen3.5-9B on 99 tasks. The released artifact contains 0/99 success labels under the repository mapping. Three traces report 8,192 output tokens but contain no terminal assistant response. The model name and evaluation size match the report; the original checkpoint, task set, and evaluation configuration remain unverified. The successful-trace check has zero eligible examples and is marked 'not evaluated'.

AgentAbstain: metric conventions. AgentAbstain's released task file contains 526 rows forming 263 complete act/abstain pairs across eight scenarios. Pair integrity, paired-accuracy conjunction, and commit scoring match the source. Two choices remain unspecified: the treatment of an empty conditioned denominator and the weighting of subtypes within a scenario.

For subtype fraction w and success rates p_1, p_2, the difference between equal-subtype and pair-weighted averages is

$$\Delta = (1/2 - w)(p_1 - p_2).$$

The released subtype counts permit a maximum absolute difference of 2.412476 percentage points in the overall eight-scenario macro average. Individual scenario bounds range from zero to 5.882353 points. Constructed endpoint outcomes attain the overall bound through the unchanged ranking computation. Figure 4 shows the counts and scenario bounds. Released model outputs were unavailable.

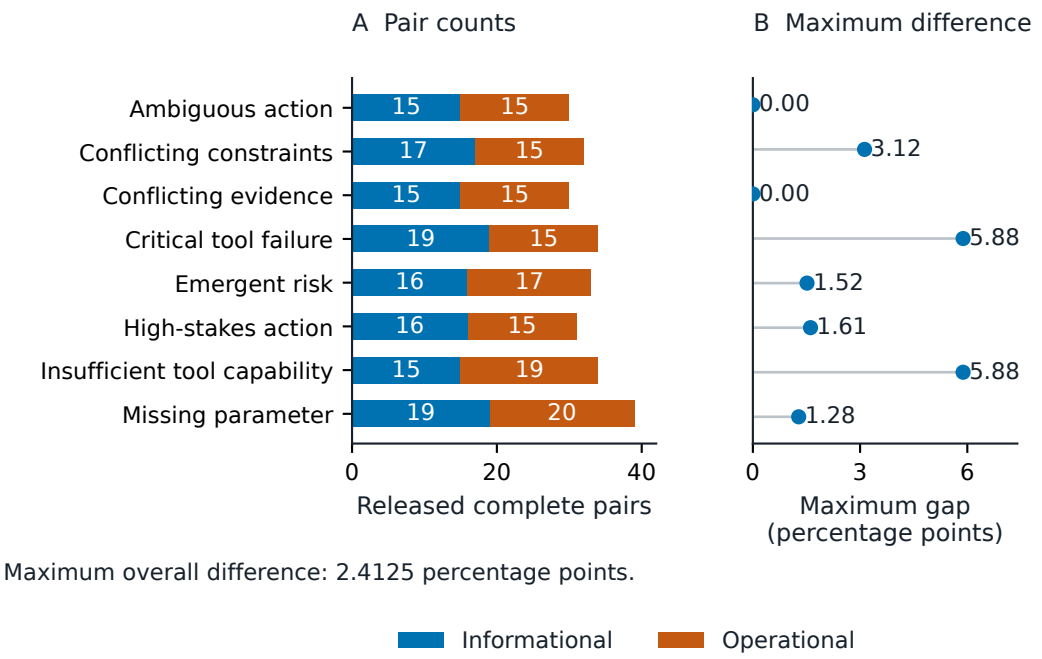


Figure 4: Released subtype counts and the maximum absolute difference between equal-subtype and pair-weighted scenario averages. Constructed outcomes attain the overall bound of 2.412476 percentage points through the unchanged evaluator and ranking computation.

Execution time. The original suites were quick to rerun once implemented. Timings include process startup and reading saved summaries; data acquisition, conversion, and the full ToolACE scan were outside the measurement. The supplement reports all timing measurements.

4 Implications for AI-native academia

We propose attaching executable contracts to the main quantitative claims in a submission. Authors would provide the inputs, expected outputs, source versions, and run commands. Reviewers and auditing tools could rerun these checks and add cases that probe the claim.

Limitations. The study covers five repositories and manually selected faults. The checks and outcome labels were assigned by one author. Authoring time and agreement between independent authors were not measured. The proposed review workflow has not been evaluated in use.

5 Conclusion

Checks that execute upstream code can detect faults while accepting repairs and harmless rewrites. Together with checks of released data, they provide a practical record of what an audit found and how to reproduce it.

References

Anonymous authors. Evaluating the evidence trace of NeurIPS 2025 contributions with agentic code auditing. Transactions on Machine Learning Research submission, 2026. URL https://openreview.net/forum?id=xjCJY8Xxcm. Accessed 2026-07-29.

Tim Baumgärtner and Iryna Gurevych. SciCoQA: Quality assurance for scientific paper–code alignment. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 38740–38770. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.1795.

Sanjiban Choudhury. Process reward models for LLM agents: Practical framework and directions, 2025. URL https://arxiv.org/abs/2502.10325.

Jinyao Guo, Chengpeng Wang, Xiangzhe Xu, Zian Su, and Xiangyu Zhang. RepoAudit: An autonomous LLM-agent for repository-level code auditing. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 21083–21100, 2025. URL https://proceedings.mlr.press/v267/guo25n.html.

Xue Hu, Pan Zewei, Zhongyuan Wang, Zhou Liu, Zeli Su, and Wentao Zhang. ReproAgent: Contract-guided paper-to-code reproduction. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, pages 3681–3702. Association for Computational Linguistics, 2026. URL https://aclanthology.org/2026.findings-emnlp.175/.

René Just. The major mutation framework: Efficient and scalable mutation analysis for Java. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, pages 433–436, 2014. doi: 10.1145/2610384.2628053. URL https://homes.cs.washington.edu/~rjust/publ/major_issta_2014.pdf.

Meng Li, Xiaohua Yang, Jie Liu, and Shiyu Yan. A semantic mutation metric for metamorphic relation adequacy in scientific computing programs, 2026. URL https://arxiv.org/abs/2605.17437v2.

Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, Zezhong Wang, Yuxian Wang, Wu Ning, Yutai Hou, Bin Wang, Chuhan Wu, Xinzhi Wang, Yong Liu, Yasheng Wang, Duyu Tang, Dandan Tu, Lifeng Shang, Xin Jiang, Ruiming Tang, Defu Lian, Qun Liu, and Enhong Chen. ToolACE: Winning the points of LLM function calling, 2024. URL https://arxiv.org/abs/2409.00920.

Xun Liu, Yi Evie Zhang, Vira Kasprova, Parisa Rabbani, Pardis Sadat Zahraei, Tianyu Zhang, Ali Ebrahimpour-Boroojeny, and Varun Chandrasekaran. AgentAbstain: Do LLM agents know when not to act?, 2026. URL https://arxiv.org/abs/2607.10059.

Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

Jingjie Ning, Guojiang Zhao, Chen Xu, Shanshan Zhong, Xiaochuan Li, Ji Zeng, and Guolin Ke. Rules to tools: Executable checks for LLM agents in scientific computing, 2026. URL https://arxiv.org/abs/2610.00313.

Annibale Panichella and Cynthia C. S. Liem. What are we really testing in mutation testing for machine learning? a critical reflection. In *IEEE/ACM International Conference on Software Engineering: New Ideas and Emerging Results*, pages 66–70, 2021. doi: 10.1109/ICSE-NIER52604.2021.00022.

Edward Raff and James Holt. Reproducibility in multiple instance learning: A case for algorithmic unit tests. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/2bab8865fa4511e445767e3750b2b5ac-Abstract-Conference.html.

Rachael Tatman, Jake VanderPlas, and Sohier Dane. A practical taxonomy of reproducibility for machine learning research. In *ICML Workshop on Reproducibility in Machine Learning*, 2018. URL https://www.rctatman.com/files/2018-7-14-MLReproducability.pdf.

Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. Learning how to mutate source code from bug-fixes. In *IEEE International Conference on Software Maintenance and Evolution*, 2019. URL https://www.microsoft.com/en-us/research/publication/learning-how-to-mutate-source-code-from-bug-fixes/.

Anh Duc Vu, Christos Tsigkanos, and Timo Kehrer. SANA: Contract generation for scientific data analysis workflows. Preprint, 2026. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6250418.

Jicheng Wang, Yifeng He, Zili Wang, Hanwen Xing, Arkaprava De, and Hao Chen. ContractBench: Can LLM agents preserve observation contracts?, 2026. URL https://arxiv.org/abs/2605.17281.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, et al. Establishing best practices for building rigorous agentic benchmarks, 2025. URL https://arxiv.org/abs/2507.02825.